

DECA Spring Term Exam Notes 2025

(These notes will be distributed in the Exam printed greyscale 2 slides per side)

- The EEP1 CPU is a 16 bit Harvard architecture, with 16 bit addresses and word addressing for both data and code, hence 2^{17} bytes of instruction memory and 2^{17} bytes of data memory.
- At start of execution all EEP1 registers and flags are zero
- All EEP1 instructions take 1 clock cycle to execute

EEP1 Machine Code Overview

SHIFTOPC(1:0)	Shift
0	LSL
1	LSR
2	ASR
3	XSR

ALUOPC	INS(8)=0	INS(8)=1
0	MOV ¹ Ra, Rb	MOV Ra, #IMM
1	ADD Rc, Ra, Rb	ADD Ra, #IMM
2	SUB Rc, Ra, Rb	SUB Ra, #IMM
3	ADC Rc, Ra, Rb	ADC Ra, #IMM
4	SBC Rc, Ra, Rb	SBC Ra, #IMM
5	AND Rc, Ra, Rb	AND Ra, #IMM
6	CMP ¹ Ra, Rb	CMP Ra, #IMM
7	Shift Ra, Rb, #SCNT (see SHIFTOPC for Shift)	

¹Instruction does not use Rc, this field is (0)

legend	Meaning
Imm4	4 bit unsigned immediate
Imms5	5 bit signed immediate
Imms8	8 bit signed immediate
(0)	Must be 0 for current instructions, non-zero values are reserved for expansion

EEP1 machine code	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	ALU	0	ALUOPC=7			a			Shift- opc(1)	b			Shift- opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c		(0)		
									1	Imms8 (IMM)							
	Load / Store	1	0	MOPC	(0)	a			0	b			Imms5 (OFFSET)				
									1	Imms8 (IMM)							
	Jump	1	1	0	0	JMPOPC				Imms8 (JOFFSET)							
	EXT	1	1	0	1	(0)				Imms8 (IMM)							
	Not used	1	1	1	X	X											

EEP1 ALU Instructions

SHIFTOPC(1:0)	Shift
0	LSL
1	LSR
2	ASR
3	XSR

ALUOPC	INS(8)=0	INS(8)=1
0	MOV ¹ Ra, Rb	MOV Ra, #IMM
1	ADD Rc, Ra, Rb	ADD Ra, #IMM
2	SUB Rc, Ra, Rb	SUB Ra, #IMM
3	ADC Rc, Ra, Rb	ADC Ra, #IMM
4	SBC Rc, Ra, Rb	SBC Ra, #IMM
5	AND Rc, Ra, Rb	AND Ra, #IMM
6	CMP ¹ Ra, Rb	CMP Ra, #IMM
7	Shift Ra, Rb, #SCNT (see SHIFTOPC for Shift)	

¹Instruction does not use Rc, this field is (0)

Legend	Meaning
Imm4	4 bit unsigned immediate
Imms8	8 bit signed immediate
(0)	Must be 0 for current instructions, non-zero values are reserved for expansion

EEP1 machine code	INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	ALU	0	ALUOPC=7			a			Shift- opc(1)	b			Shift- opc(0)	Imm4 (SCNT)			
			ALUOPC=0..6			a			0	b			c		(0)		
									1	Imms8 (IMM)							

EEP1 ALU instructions: detail

OPCALU	Mnemonic	ALU.Out: INS(8)=0	ALU.Out: INS(8) = 1	Write FlagV & FlagC ¹	Write register	
0	MOV	Rb	Imm	No	Ra	move
1	ADD	Ra + Rb	Ra + Imm	Yes	Rc (Ra if INS(8)=1)	arithmetic
2	SUB	Ra - Rb	Ra - Imm	Yes	Rc (Ra if INS(8)=1)	
3	ADC	Ra + Rb + C	Ra + Imm + C	Yes	Rc (Ra if INS(8)=1)	
4	SBC	Ra - Rb + (C-1)	Ra - Imm + (C-1)	Yes	Rc (Ra if INS(8)=1)	
5	AND	Ra & Rb	Ra & Imm	No	Rc (Ra if INS(8)=1)	"bitwise" logical
6	CMP	Ra - Rb	Ra - Imm	Yes	no	comparison
7	LSR, ASR, XSR, LSL	depends on SHIFTOPC		Yes ²	Ra	shift

Flag	
FlagN	z(ALU.Out) < 0
FlagZ	ALU.Out = 0
FlagC	carry
FlagV	signed overflow

&	16 bit C++ bitwise logical AND operator
---	---

FlagN & **FlagZ** are written by **all** ALU instructions

Flags are only written by ALU instructions

¹FlagC

- 1 bit flip-flop in CPU used to store carry for multi-word addition and shifts
- Holds previous value unless written

C = FlagC.Q

²**Shift instructions** write FlagC with *last bit shifted out*, FlagV is written to an undefined value.

EEP1 Memory Instructions

LDR	Load Register
STR	Store Register

Field	Limits	Allowed Syntax
Imms5	-16 .. +15	#0xA, #12, #-0x10, #-15
Imms8	-128 .. 127	#0x34, #100, #-33
a, b	0..7	R0, R5, R7

INS bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	0	MOPC	(0)	a			0	b			Imms5 (OFFSET)				
								1	Imms8 (IMM)							

Type	MOPC	I(8)	Assembly Syntax	Examples	Effective Address	Operation
register offset load	0	0	LDR Ra, [Rb, #Imms5] ³ LDR Ra, [Rb] ²	LDR R4, [R6, #-12] LDR R2, [R7]	Rb+Imms5 ¹ Rb	Ra := mem[Rb + Imms5 ¹] Ra := mem[Rb]
direct load	0	1	LDR Ra, [#Imms8] ⁴	LDR R0, [#220]	Imms8 ¹	Ra := mem[Imms8 ¹]
register offset store	1	0	STR Ra, [Rb, #Imms5] ³	STR R6, [R6, #9]	Rb+Imms5 ¹	mem[Rb + Imms5 ¹] := Ra
direct store	1	1	STR Ra, [#Imms8] ⁴	STR R0, [#0x66]	Imms8 ¹	mem[Imms8 ¹] := Ra

¹ Imms5 is a 5 bit 2's complement signed number sign extended to 16 bits, Imms8 is 8 bits sign extended to 16 bits. Imms8=255 is the same as Imms8=-1.

² Special case: assembly syntax allows Imms5 offset to be omitted in which case it has value 0 in the machine code.

³ Register offset addressing. ⁴ Direct addressing.

EEP1 Jump instructions

EEP1 machine code	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	1	0	0	JMPOPC				Imms8 (JOFFSET)							

JMPOPC(3:1)	NZCV condition	Condition on data for jump with JMPOPC(0)=0	JMPOPC(0) = 0	JMPOPC(0) = 1 ³
0	1	always	JMP (Always jump)	NOP (Never jump)
1	<i>Z</i>	result = 0	JEQ (Equal)	JNE (Not Equal)
2	<i>C</i>	$C=1 \equiv u(Ra) \geq u(Op)$ ¹	JCS (Carry Set / Unsigned Higher or Equal)	JCC (Carry Clear / Unsigned Lower)
3	<i>N</i>	$z(result) < 0$	JMI (Minus)	JPL (Plus)
4	$\bar{N} \oplus V$	$z(Ra) \geq z(Op)$ ¹	JGE (Signed Greater or Equal)	JLT (Signed less than)
5	$(\bar{N} \oplus V) \cdot \bar{Z}$	$z(Ra) > z(Op)$ ¹	JGT (Signed Greater Than)	JLE (Signed Less than or Equal)
6	$C \cdot \bar{Z}$	$u(Ra) > u(Op)$ ¹	JHI (Unsigned Higher)	JLS (Unsigned Lower or Same)
7	1	always	JSR ($R7^4 := PC + 1, PC := PC + JOFFSET$)	RET ($PC := R7$)

¹ **JGE,JGT,JHI,JCS** jump conditions jump based on the flag values. The table column 3 shows the comparison of **Ra** and **Rb** after a previous **SUB Ra, Rb** instruction which delivers those NZCV values, or the condition on an ALU result which delivers the required NZ values.

² **JSR** and **RET** always jump and are used to *jump to* and *return from* subroutine see notes. **RET** does not use **Joffset**.

³ **JMPOPC(0) = 1** inverts all conditions, except for JSR/RET

⁴ Hardware is simpler because **JSR/RET** write/read R7 since **Ra** is normally read and written and for these instructions **a = JMPOPC(3:1) = 7**

Jump Instructions	Assembly	Operation	Examples
All except RET, JSR	JEQ JOFFSET; JEQ SYMBOL	$PC := PC + JOFFSET$ if condition is true	JMP -1 ; JGT LOOP ; JSR SUB1
RET	RET	$PC := R7$	RET
JSR	JSR JOFFSET	$R7 := PC+1; PC := PC + JOFFSET$	JSR 10

EXT instruction

EEP1 machine code	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	1	1	0	1	(0)				Imm8 (IMM)							

Assembly example

Memory
address

0x0100 EXT 0x23
 0x0101 JMP 0xF0 ;modified jump instruction
 0x24F1 ...

jump

- ❖ EXT modifies the instruction immediately after it, replacing bits (15:8) of Imm16 (otherwise from Imms8 sign extension) with IMM from the EXT instruction
- ❖ NB this means the 8 Imms8 bits from the modified instruction are not sign extended because the extended part is replaced.
- ❖ Applies to:
 - Jump instructions,
 - **Direct** memory instructions
 - ALU instructions with immediate operand

0x2300 EXT IMM8 = 0x23

0x00F0 JMP IMMS8 = 0xF0

0x0101 Address (PC) of JMP instruction

0x24F1 Jump target address

EEP1 Subroutine Call Protocol determines how registers are used by subroutines

- ❖ The EEP1 subroutine stack is a *downwards growing, empty stack pointer* stack using R6 as stack pointer.
- ❖ The protocol fixes the usage of each register for all subroutines and determines what must be saved or restored on subroutine entry/exit.
 - **Scratch** registers (can be changed by any subroutine, not saved/restored at entry/exit)
 - **Persistent** registers (must be saved/restored at entry exit by callee subroutine on callee's stack frame if used)
 - **Stack Pointer**
 - **Return Address**
- ❖ All ISAs use protocols similar to this so that compiled code can be turned into subroutines
- ❖ Scratch registers are also used if needed for 1st three subroutine inputs and outputs

EEP1 standard subroutine call protocol (the <i>caller</i> calls the <i>callee</i>)			
Register	Content Type	Saved by callee on subroutine entry/exit if used	Persistent across subroutine calls, can be used for storage by caller
R7	Return Address	yes	no
R6	Stack Pointer (SP)	n/a	yes, no*
R5	persistent	yes	yes
R4	persistent	yes	yes
R3	persistent	yes	yes
R2	scratch (or input 3, or output)	no	no
R1	scratch (or input 2, or output)	no	no
R0	scratch (or input 1, or output)	no	no

*R6 can't be used for storage by caller, because it must always contain the stack pointer

Notation for DECA Spring 2025 (1)

- **High, low.** Synonym for digital logic values 1, 0.
- **X.** *don't care* value that does not affect operation
- **Clocked (sequential) logic.** Logic with edge triggered flip-flops. All clocks are assumed connected together to a single global clock.
- **Combinational logic.** Logic with gates and no flip-flops, also no *latches* AKA RS flip-flops, nor logic with feedback connections.
 - NB we will not use *latches* in this module, therefore combinational logic loops (that might make latches) are also not allowed. Every such loop must be broken by a D -> Q flip-flop path, or equivalent memory path.
- **Asynchronous (synchronous) RAM.**
 - The *output* is valid in the same cycle (the next cycle) as the *address*. In both cases the write operation is synchronous, and locations change to a newly written value in the cycle after that in which WEN is 1 and ADDR, DIN specify the written location and value.
- **Register EN** – enable input, if 1 then something is written
 - Named WEN on RAMs and register files
- **Decoder EN** – enable input, if 0 then all outputs are 0
 - Note differing use of EN on clocked and combinational logic.
- **A := B.** Notation to set register A equal to B on active clock edge at end of current clock cycle. B can be a register or a number.
- **Mem[a]** – memory *location* with *word* address a. **Mem[a] := B** – set memory location address a to contents of B (or B – if B is a number).
- **Mem₃₂[a] (Mem₁₆[a]), Mem₈[a]** word and byte locations in a 32 (16) bit system with byte addressing such as ARM. Word or byte memory read and write are defined with := notation as above for Mem[a]. Byte memory transfers are as defined in notes.
- **MUX.SEL** – signal SEL on block MUX
- **MUX.N** input – selects which of MUX data inputs passes to output. Data inputs are numbered according to their N value.
- **IR(7)** – *selected bit 7* of binary data or register IR. Bits of n bit bus are numbered *little-endian* with bit 0 being LSB, bit n-1 MSB.
- **IR(3:0)** – *slice* of binary data or register IR. Bits from 3 to 0 inclusive.

Notation for DECA Spring 2025 (2)

- **Byte.** 8 bits of data. Possibly interpreted as unsigned (0..255) or signed (-128..127) number. See $u(A)$ & $z(A)$ definitions
- **Word.** Binary data in one CPU memory location or CPU data register. Typical values 8, 16, 32, 64 bits. Sometimes interpreted as unsigned or signed number.
- **MSW (LSW)** Most (Least) Significant Word, when words are joined to make a binary number with more bits
- **MSB (LSB).** **MS byte, LS byte.** The most significant or least significant bit or byte respectively of a binary word.
- **R1:R2.** The binary value made from the bits of R1 (MSBs) and R2 (LSBs). If R1, R2 are 16 bit: $(R1:R2)(31:16) = R1$ and $(R1:R2)(15:0) = R2$
- **A_N** - Value of the **A** signal in cycle N. NOT **A(N)** the Nth bit of A!
- **0xA30, 0b010**, hex, binary representations of numbers.
- **R0, Rn** CPU register 0, n (e.g. $n = 0..3$ or $n = 0..15$). CPU registers have numbers R0, R1, ... For a register file with named registers REGn REG[i] indicates register REGi.
- **$u(A)$, $z(A)$** , Respectively the *unsigned* and *signed* numeric interpretation of signal A. A must have a known width n , in which case $u(A)$ and $z(A)$ differ only when the sign bit $A(n-1)$ is 1. *Bit slicing* is allowed on integers derived from $u()$, $z()$: $(u(P(n-1:0)) + u(Q(n-1:0)))(n-1:0)$ is the n Sum bits of a binary adder with n -bit inputs P, Q. The carry output bit from this could be written as: $(u(P(n-1:0)) + u(Q(n-1:0)))(n)$
- **A_s** = $A(n-1)$, the most significant (sign) bit of $A(n-1 : 0)$.
- **A'** If $A = A(n-1 : 0)$ has n bits, **A'** = $A(n-2 : 0)$. If used in a numeric context this is interpreted as $u(A')$: an unsigned integer.
- **$m \bmod n$** . The same as $m \% n$ in C++. **$m \bmod 2^n$** is the unsigned value of the LS 8 bits of m . If m is negative this is two's complement: $-1 \bmod 2^8 = 255$; $260 \bmod 2^8 = 4$; $x \bmod 2^n$ is x truncated to its n least significant bits. Division is assumed integer unless otherwise stated.